

Lecture Unit 6.1

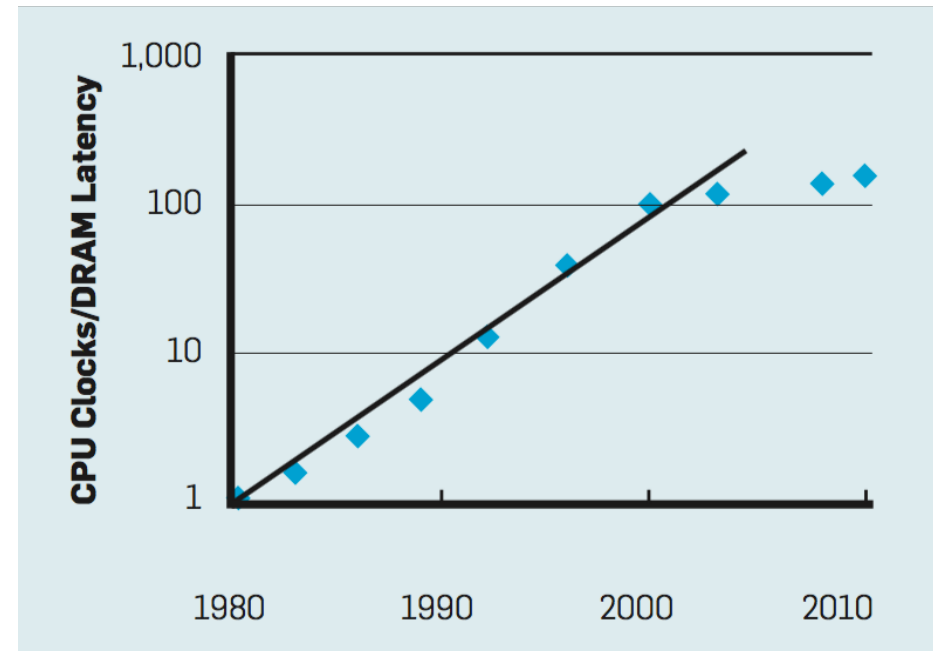
Memory hierarchy and cache

Introduction

- When performing calculations we need to read data from memory and write data back to memory (loads and stores)
- In modern computers the way memory is used has a significant impact on performance
- In this unit we will look at the memory hierarchy and its impact on performance

Memory access vs CPU speed

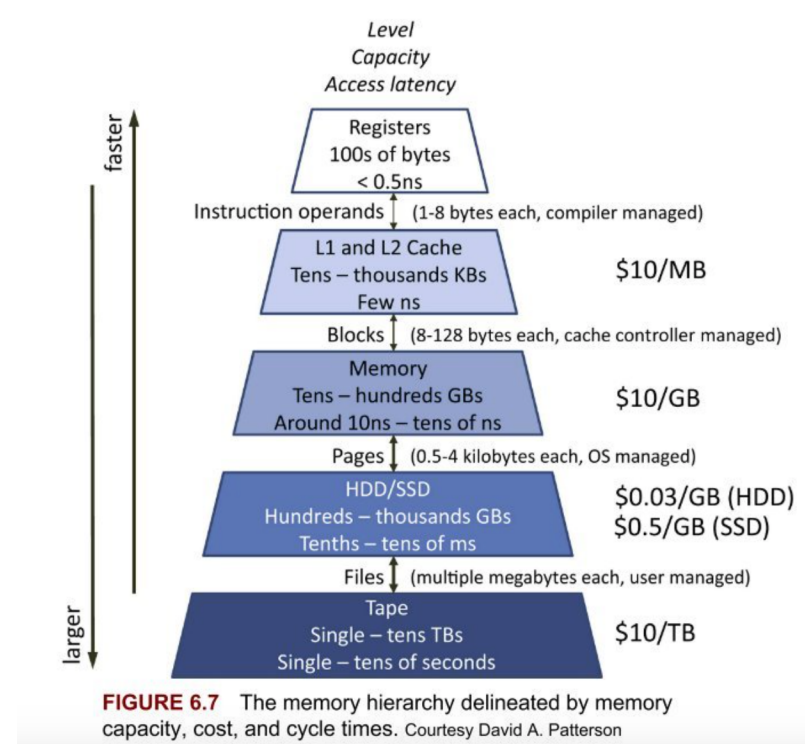
Memory is slow compared to CPU speed: ~100 clock cycles to access main memory.



[Borkar and Chein, 2011, Communications of the ACM, Vol. 54 No. 5, Pages 67-77, doi:10.1145/1941487.1941507](#)

Memory access vs CPU speed

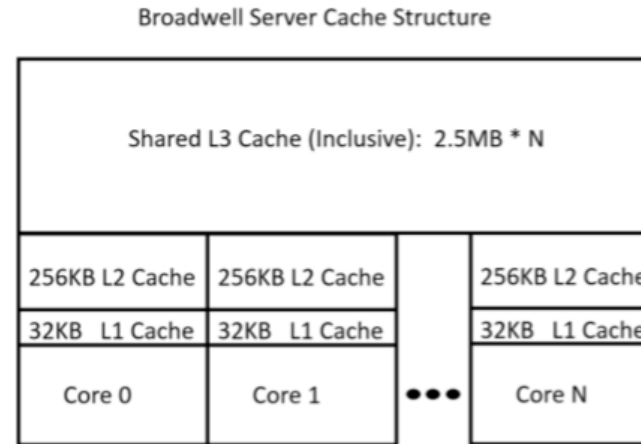
- Data can be stored in different levels of the memory hierarchy
- The memory hierarchy provides a balance between capacity, cost and access time
- Cache is a small amount of fast memory which holds data fetched from and written to main memory



SAB figure 6.7

Cache hierarchy

- Modern CPUs have multiple levels of cache
- 16-core nodes on Isca have 2x8 core processors:
 - L1=32kB (4 cycle latency)
 - L2=256kB (11 cycle latency)
 - L3=20MB (~34 cycle latency)
- `/proc/cpuinfo` reports L3 cache size



8x 2.5MB

Data from <https://software.intel.com/sites/default/files/managed/9e/bc/64-ia-32-architectures-optimization-manual.pdf> Latencies are fastest latencies from table 2-16. Cache diagram is from figure 2.6.

Temporal and spatial locality

- Cache helps hide memory latency provided we make efficient use of the data it holds
- Cache relies on the predictability provided by **locality of reference**:
 - Data nearby in memory is more likely to be used (**spatial locality**)
 - Data will be re-used within a relatively short time window (**temporal locality**)

Spatial locality

- Data is transferred from main memory to cache in fixed size blocks called cache lines
- Cache lines on Intel x86_64 processors are 64 bytes (512 bits)
- When we access adjacent memory locations the next piece of data required is likely to be in cache already (a cache hit)
- When the next piece of data is not in cache it needs to be loaded from main memory (cache miss)
- Read from memory locations sequentially to use the whole cache line (access arrays stride one)

Memory access example

memory1.c

```
#include <stdio.h>
#define N 40000

float data[N][N];

int main(){
    int i, j;
    for (j=0; j<N; j++){
        for (i=0; i<N; i++){
            data[i][j] = (float) (i*j);
        }
    }

    /* Print a value to stop the compiler
       optimising loops away */
    printf("Val= %f\n", data[10000][10000]);

    return 0;
}
```

memory2.c

```
#include <stdio.h>
#define N 40000

float data[N][N];

int main(){
    int i, j;
    for (i=0; i<N; i++){
        for (j=0; j<N; j++){
            data[i][j] = (float) (i*j);
        }
    }

    /* Print a value to stop the compiler
       optimising loops away */
    printf("Val= %f\n", data[10000][10000]);

    return 0;
}
```

- The two versions of the program differ only in the order of i and j in the loop nest
- In C the j loop should be on the inside so that memory is accessed sequentially

Spatial locality

Compare the two programs

Use the `time` command to measure the time taken to run:

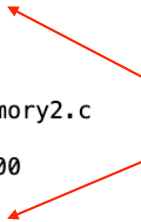
```
> gcc -o memory1 memory1.c
> time ./memory1
Val= 100000000.000000
```

```
real    0m18.012s
user    0m16.343s
sys     0m1.655s
```

```
> gcc -o memory2 memory2.c
> time ./memory2
Val= 100000000.000000
```

```
real    0m4.611s
user    0m3.096s
sys     0m1.509s
>
```

The second version is
3.9 times faster



The array is 40000x40000 4-byte floats

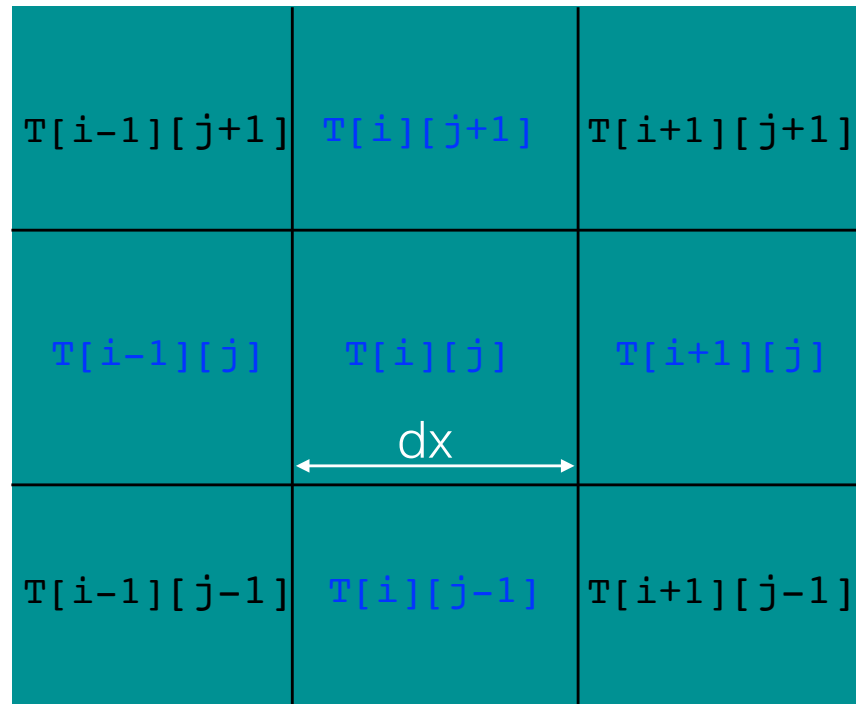
$$40\,000^2 \times 4 = 6.4 \times 10^9 \text{ bytes}$$

Much larger than the 8MB cache.

The loop order matters for writing too.

Finite differences and cache

- Finite differences in the j direction access memory stride one (spatial locality)
- Finite differences in the i direction access array elements not stored nearby in memory
- These array elements will be in a different cache line



Cache blocking

- We can split the work up into cache-sized blocks and work on one block before moving on to the next one
- All the data required can be held in cache so can be accessed much faster
- This exploits temporal locality: re-use data in the block within a short time window
- See <https://www.intel.com/content/www/us/en/developer/articles/technical/loop-optimizations-where-blocks-are-required.html> for more on cache blocking

Size of test cases

- When debugging a program we work with a small test case which runs in a manageable time
- Reducing the problem size can change the performance characteristics in a non-uniform manner (e.g. if a significant data structure fits into cache)
- When testing performance we need a test case with a representative memory footprint
- For example when testing the performance of a PDE solver use a representative domain size but run for fewer time steps

Compiler optimisations

- Compilers can optimise a program by making changes to the source code to produce equivalent results faster
- These optimisations can include optimisation of memory use e.g. loop interchange and cache blocking
- The optimisation level can be controlled with a command line flag (`-O3` turns on all optimisations in `gcc`)
- See <https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html> for more on GCC optimisations

Unit Summary

- Accessing main memory takes ~ 100 CPU cycles but this latency can be hidden by the memory cache
- Achieving good performance on modern processors requires exploiting locality of reference to make efficient use of cache:
 - **Spatial locality:** order of loop nesting matters
 - **Temporal locality:** cache blocking
- Test cases for performance analysis should use a realistic problem size as performance can scale non-uniformly with memory footprint
- See SAB section 6.5 “Memory Hierarchy”